PROVIDER_COVERAGE

HelixLLM Extended Provider Coverage — Design Mapping (P4 provider adapters)

Status	DESIGN (spike before implementation, §11.4.6 — do not scaffold an adapter until its API row here is confirmed)
Scope	The **extended LLM provider set** from the programme request, mapped into the existing LLMsVerifier provider registry (verification) + HelixLLM/HelixAgent provider layer (serving). NOT the already-integrated CONST-039 set except where an operator decision routes a new model onto it.
Owns	Implementation-plan **P4-T1** / **P4-T2** / **P4-T3** (`04_implementation_plan.md:88-92`) — the provider-adapter + protocol phase (parallel after P2)
Created	2026-07-06 · Revision 1 · Track (`T1/main`) · Branch `feature/helixllm-full-extension`
Grounding	`submodules/helix_llm/docs/API_CONTRACT.md` §2 (route inventory) §4 (OpenAI-compat) §5 (Anthropic-compat) · `submodules/llms_verifier/llm-verifier/providers/model_provider_service.go` (`ProviderClient{Name,BaseURL,APIKey}`) · `docs/research/07.2026/00_master/03_open_clarifications.md` (operator resolutions C1–C4 + resolved-without-operator) · `docs/research/07.2026/00_master/`

	`04_implementation_plan.md` P2-T1/P2-T4/P4-T1..3

> **Anti-bluff (§11.4.6 / §11.4.99 / §11.4.150).** Every base URL, auth model, and OpenAI-compat verdict below was verified against the provider's **LATEST live documentation on 2026-07-06** (WebFetch/WebSearch — never memory); each row cites the URL it was read from. Where a base URL could **not** be confirmed from a fetched authoritative page in this pass it is recorded `UNVERIFIED — <reason>` (§11.4.6) and **no endpoint is invented**. The operator decisions in `03_open_clarifications.md` (C1–C4) are honoured **verbatim** and cited, not re-litigated.

---

0. The unit of work — why almost all of this is config, not code

HelixLLM's gateway is **already OpenAI-compatible AND Anthropic-compatible** on the wire: `POST /v1/chat/completions` (`API_CONTRACT.md` §2 → `internal/gateway/router.go:72`), `GET /v1/models` (`router.go:74`, returns the provider layer's models — **not a hardcoded list**, aligning CONST-036 per `API_CONTRACT.md` §4.3), and `POST /v1/messages` Anthropic surface (`router.go:84`). The LLMsVerifier provider model is a **data record** — `ProviderClient{ Name, BaseURL, APIKey }` (`providers/model_provider_service.go:45-49`) — whose model set is discovered by querying the provider's own `/v1/models`, never enumerated in source (CONST-036).

The programme's own plan states the consequence: P4-T1 adapters land in **HelixAgent `internal/llm/providers/` + an LLMsVerifier registration** (`04_implementation_plan.md:90`), and P2-T1 already extends "HelixAgent's OpenAI/Anthropic REST server as the unified `/v1` gateway … **reuse not reimplement §11.4.74**" (`04_implementation_plan.md:70`). Therefore **an OpenAI-compatible provider is a config row, not an adapter**: same request struct, same response struct, only `{Name, BaseURL, APIKey-from-env}` differ (§11.4.28 decoupling / §CONST-045 no-hardcoded-host / §CONST-046 no-hardcoded content). A **new adapter** is only warranted when a provider's wire shape is NOT OpenAI-compatible (none of the confirmed extended providers below are in that class — see §2).

---

1. Per-provider coverage table (LATEST-doc-verified 2026-07-06)

Auth model is **Bearer API key from an env var** (`Authorization: Bearer $<PROVIDER>_API_KEY`) for **every** hosted row — the single-source-of-truth credential rule (§11.4.10, never hardcoded/logged); the column is elided as uniform and noted only where a provider adds a second surface.

1.1 Confirmed hosted providers — OpenAI-compatible, drop into the existing adapter (config-only)

Provider	Base URL (verified)	OpenAI-compat? → reuse	Stat
Poe	`https://api.poe.com/v1`	**Yes — reuse** (chat/completions + responses)	**GA**
Perplexity (Sonar)	`https://api.perplexity.ai`	**Yes — reuse** (`/chat/completions`, Bearer)	**GA**
Sakana Fugu	`https://api.sakana.ai/v1`	**Yes — reuse** (Chat Completions, Responses, Models)	**GA**
Xiaomi MiMo	`https://api.xiaomimimo.com/v1`	**Yes — reuse** (OpenAI **and** Anthropic surfaces)	**GA**
Tencent Hunyuan		**Yes — reuse** (drop-in OpenAI SDK)	**GA**

Provider	Base URL (verified)	OpenAI-compat? → reuse	Stat
(Yuanbao = consumer app → API target Hunyuan)	`https://api.hunyuan.cloud.tencent.com/v1`		
xAI Grok	`https://api.x.ai/v1`	**Yes — reuse** (OpenAI client; xAI also documents an Anthropic surface)	**GA**
Moonshot / Kimi	`https://api.moonshot.ai/v1`	**Yes — reuse**	**GA**
Zhipu / Z.ai GLM	`https://api.z.ai/api/paas/v4/`	**Yes — reuse** (OpenAI Python/ Node SDK)	**GA**
Fireworks AI	`https://api.fireworks.ai/inference/v1`	**Yes — reuse**	**GA**
DeepInfra	`https://api.deepinfra.com/v1/openai`	**Yes — reuse**	**GA**
Novita	`https://api.novita.ai/v3/openai`	**Yes — reuse** (ChatCompletion + Completion)	**GA**
AI21 (Jamba)	`https://api.ai21.com/studio/v1`	**Yes — reuse, with caveat** (OpenAI-shaped `/chat/completions` + AI21 superset field `documents`)	**GA**
Reka	`https://api.reka.ai/v1`	**Yes — reuse** (`OpenAI(base_url="https://api.reka.ai/v1")`)	**GA**

Confirmed OpenAI-compatible hosted adapters (config-only reuse): 13.

1.2 Base-URL-unverified this pass — verify-at-build before scaffolding (§11.4.6)

Provider	Best-known base URL	Status	Why UNVERIFIED	Ac
Hyperbolic	*(commonly* `https://api.hyperbolic.xyz/v1`*)*	GA (believed)	**UNVERIFIED — not fetched from an authoritative page in this pass.** `04_implementation_plan.md:90` already says “verify Hyperbolic/ Baseten base URLs at build”	Fe Hy AP pa if /v co Cl ro th
Baseten	*(per-deployment Model API host, not one global base)*	GA	**UNVERIFIED — Baseten exposes per-model/per-deployment endpoints**, so there is no single global base URL to hardcode; its Model APIs are OpenAI-compatible per deployment	Tr UR **de co** (§ re Op ad a d de UR in

1.3 Operator-decided mappings — honoured verbatim (`03_open_clarifications.md` C1–C4)

Item (as given)	Operator resolution (2026-07-06) — cited	Layer	Adapter action
GPT-Sol (C1)	**OpenAI GPT-5.6 “Sol”** — “NO new adapter; add model IDs to the existing OpenAI provider when it reaches GA (track as preview-pending).”	Provider (existing OpenAI adapter)	**No new adapter** (P4-T2). Add the `gpt-5.6-sol` model IDs to the existing OpenAI `ProviderDescriptor` **at GA**; carry a `preview-pending` flag until then

Item (as given)	Operator resolution (2026-07-06) — cited	Layer	Adapter action
Google OKF (C2)	**RAG/Skills knowledge format via MCP resources** — "implement OKF as the on-disk knowledge format served through an MCP `resources` server (context layer, not provider layer)."	**Context / MCP resources — NOT a provider**	**Out of provider-coverage scope.** No LLM adapter. Belongs to P3-T7 RAG + the MCP `resources` server (stream 05). Recorded here so it is not mistaken for a missing provider
ACP (C3)	**Google A2A (Agent-to-Agent)** — "implement A2A interop, NOT Zed ACP."	**Protocol (agent ↔ agent) — NOT a provider**	**Out of provider-coverage scope.** A2A interop is a protocol surface (P4 protocols), served alongside `/v1/models` + MCP `server/discover`; not a model endpoint
Subquadratic (SubQ) (C4)	**BLOCKED-until-GA** — "no public base URL/auth … no adapter work possible until a public API (or operator-supplied beta credentials) exists."	Provider (deferred)	**BLOCKED-until-GA.** `UNVERIFIED — private beta, no public base URL/auth (operator C4).` Tracked deferred item; **no endpoint invented**, no acceptance test until GA

1.4 Self-host mappings — reuse the existing local path, no hosted adapter (`03_open_clarifications.md` + P4-T3)

Item	Resolution (cited)	Adapter action
Qwythos 9B	`empero-ai/Qwythos-9B-…` (Qwen3.5-9B, 1M ctx, Apache-2.0) — **self-host via the existing local/ Ollama/HF path, no new hosted**	Served through the **already-integrated Ollama / Llama.cpp / local HF path** (CONST-039). A model-config

Item	Resolution (cited)	Adapter action
	adapter (`03_open_clarifications.md`; P4-T3)	addition, not a provider adapter
GOT (GOT-OCR2.0)	**An OCR model, not a chat LLM** — "belongs to the OCR capability (stream 07), self-host" (`03_open_clarifications.md`; P4-T3)	**Out of provider-coverage scope** — it is an OCR engine option under P3-T8 (`/v1/ocr`), verified by the OCR conf-floor + ROI signature, not a chat round-trip

> **Note on "GOT/GPT-Sol" in the request phrasing (§11.4.6):** these are **two distinct items**, not one — **GPT-Sol** is the OpenAI GPT-5.6 "Sol" model (C1, existing OpenAI adapter), while **GOT** is GOT-OCR2.0 (an OCR engine, P3-T8). They are separated here so neither is silently dropped.

---

2. Integration approach — reuse vs new adapter, and where each is wired

2.1 Reuse the existing OpenAI-compatible adapter (config-only) — the default

All **13 confirmed hosted providers (§1.1)** and **GPT-Sol on the existing OpenAI adapter (§1.3)** are integrated with **zero new adapter code**. Each is a `ProviderDescriptor` row supplying `{Name, BaseURL, APIKey-env-var}` and an optional per-provider header/quota tweak (Hunyuan's 5-concurrent default; Z.ai's `/api/paas/v4/` path). The wire request/response is the existing `api.ChatCompletionRequest` / `api.ChatCompletionResponse` (`API_CONTRACT.md` §4.1). This is the §11.4.74 extend-don't-reimplement + §11.4.28 decoupling posture: a provider is **data**, not a class.

- **AI21 caveat:** OpenAI-shaped but carries a superset `documents` field. Reuse the adapter as-is (the extra field is inert to a plain client); *only if* RAG-doc grounding through AI21 is wanted does `api.ChatCompletionRequest` gain an additive, **reviewed** field (§11.4.6 — a flagged diff, never ambient).

2.2 New adapter needed — count: 0 among confirmed extended providers

None of the confirmed extended chat providers require a bespoke wire adapter — every one exposes an OpenAI `/chat/completions` surface. (Several — Xiaomi, Zhipu, xAI, Moonshot — *additionally* expose an Anthropic surface that could ride

the existing `/v1/messages` path, but that is a bonus, not a requirement.) The only genuinely non-OpenAI mappings (**OKF**, **A2A**) are **not provider-layer at all** and are explicitly out of scope per the operator decisions.

2.3 Where each is wired — two registries, one per concern

1. **LLMsVerifier provider registry (verification side).** Each provider is registered as a `ProviderClient{Name, BaseURL, APIKey}` (`providers/model_provider_service.go:45-49`) so LLMsVerifier can probe it, discover its model set from the live `/v1/models` (CONST-036 — **no hardcoded model list**), and set each CONST-040 capability flag from ONE real probe with a captured wire artefact (P2-T3 fail-closed; `04_implementation_plan.md:72`). Freshness/staleness (CONST-037 24h re-queue, CONST-038 ≤60s poll) binds on the existing `event_stream` seam (P2-T4; `04_implementation_plan.md:73`).
2. **HelixLLM / HelixAgent provider layer (serving side).** The adapter row lands in HelixAgent `internal/llm/providers/` and is surfaced through the unified `/v1` gateway (P2-T1) so `/v1/models` advertises it and `/v1/chat/completions` routes to it. Capability flags surfaced to clients come **from the verifier** on the three CONST-040 channels (`/v1/models[].capabilities`, MCP `server/discover`, A2A `initialize`) per `03_open_clarifications.md`.

Model IDs, base URLs, keys, and capability flags are therefore **all config/verifier-sourced** — CONST-036/037/040 + §CONST-045/046 hold by construction; adding a provider is a config + verifier-registration change, no model literal in source.

---

3. §11.4.108 acceptance — the machine-checkable proof per provider class

“Registered” ≠ “reachable” ≠ “a model really answered.” Each class below declares ONE machine-checkable **runtime signature** on a clean target (§11.4.108/§11.4.139), captured to `docs/qa/<run-id>/provider_coverage/`. Credential-absent and BLOCKED-until-GA providers are an **honest SKIP-with-reason (§11.4.3), never a faked PASS** — the P4-T1 plan already binds “live round-trip proof (§11.4.69)” per adapter (`04_implementation_plan.md:90`).

Class A — confirmed OpenAI-compatible hosted (§1.1) + GPT-Sol-at-GA

> **RUNTIME SIGNATURE (real generate round-trip):** with the provider’s key present, POST an **unforgeable challenge** to `POST /v1/chat/completions` against the live base URL — a prompt whose correct

answer the harness computes independently (e.g. a fresh arithmetic pair, or "echo this nonce <uuid>") — and assert **all** of: (1) HTTP 200 + OpenAI-shaped body; (2) `choices[0].message.content` is **non-empty and contains the expected answer/nonce** (kills a stubbed/echoing/empty response — the §4.4-class bluff); (3) the `model` field echoes a model that also appears in the **live `GET /v1/models`** (proves discovery is not hardcoded — CONST-036); (4) the verifier recorded the CONST-040 capability flags from this same wire artefact. Captured artefact = the raw request+response JSON + the live `/v1/models` list + PASS/FAIL verdict with its evidence path. **Credential absent** → `ab_skip_with_reason … feature_disabled_by_config` (§11.4.3/§11.4.69), never PASS. Re-runnable at `-count=3` (§11.4.50/§11.4.98). **Self-validation (§11.4.107(10)):** golden-good = a captured real answer → analyzer PASS; golden-bad = empty-content / wrong-nonce / hardcoded-model-not-in-`/v1/models` responses → analyzer MUST FAIL. Paired §1.1 mutation: strip the nonce-match/non-empty assertion → the empty-content golden-bad PASSes → gate FAILs.

Class B — operator-mapped onto the existing OpenAI adapter (GPT-Sol, C1)

Same Class-A signature, **but** gated on GA. Until GPT-5.6 "Sol" is GA the row is `preview-pending` and the test **SKIPs with reason `feature_disabled_by_config` / not-yet-GA** (§11.4.3). At GA the signature adds one assertion: the `gpt-5.6-sol` id appears in the live OpenAI `/v1/models` **and** completes the round-trip — proving the model-ID addition actually reached the deployed adapter (§11.4.108 SOURCE→ARTIFACT→RUNTIME).

Class C — self-host via the existing local path (Qwythos 9B; GOT-OCR2.0)

Qwythos: the Class-A round-trip signature but against the **local Ollama/Llama.cpp/HF endpoint** (no hosted key). GOT-OCR2.0 is **not chat** — its acceptance is the **P3-T8 OCR signature** (per-word confidence floor + ROI on a fixture, feeding §11.4.117/§11.4.137), not a chat round-trip. Both reuse the already-integrated CONST-039 local path, so "credential absent" is not a SKIP reason here — the model file being fetched (§11.4.77) is.

Class D — BLOCKED-until-GA (Subquadratic, C4)

No acceptance test is authorable — there is no public base URL/auth to round-trip against. The item is `OPERATOR-BLOCKED` (§11.4.21) with

the honest reason “no public API (operator C4)”. It is **not** a SKIP-reported-as-PASS and **not** a fabricated endpoint (§11.4.6). Unblock choices: **[A]** operator supplies beta credentials + base URL → promote to Class A; **[B]** wait for public GA; **[C]** drop from scope if the beta closes. No adapter, no test, until [A] or [B].

Class E — not provider-layer (OKF via MCP resources, C2; A2A, C3)

Out of provider-coverage acceptance. OKF’s proof is an MCP `resources` round-trip (a knowledge doc served + retrieved — P3-T7/ stream 05); A2A’s proof is an agent↔agent `initialize` + task hand-off (P4 protocols). Neither is a `/v1/chat/completions` signature. Listed so a reviewer confirms they are **deliberately** absent from the chat-provider acceptance matrix, not forgotten.

Class F — base-URL-unverified (Hyperbolic, Baseten §1.2)

No PASS until the base URL is verified from a live authoritative page at build (§11.4.6). Once confirmed OpenAI-compat → they become Class A. Baseten additionally needs a concrete per-deployment URL injected before any round-trip. Until then the row carries no green result.

---

4. Prioritized rollout order (cheapest-real-value first)

Ordering rule: **OpenAI-compatible config-only reuse before anything needing new code or new decisions**; within reuse, most-load-bearing / operator-named first; blocked and out-of-scope last. This is the §11.4.82 iteration-speedup + §11.4.42 priority-first posture applied to provider onboarding.

1. **Wave 1 — confirmed reuse, operator-named extended providers** (config rows, Class-A signature): **Poe, Perplexity, Sakana Fugu, Xiaomi MiMo, Tencent Hunyuan, xAI Grok**. Highest programme salience, zero adapter code, all base URLs verified. (§1.1)
2. **Wave 2 — confirmed reuse, high-value additions** (P4-T1 remainder): **Moonshot/Kimi, Zhipu/Z.ai GLM, Fireworks, DeepInfra, Novita, Reka**, then **AI21** (last in wave for its `documents` superset caveat). All Class-A. (§1.1)
3. **Wave 3 — operator-mapped onto existing adapter: GPT-Sol** model-ID addition to the OpenAI provider, shipped `preview-pending`, flips to Class-A PASS at GA (P4-T2 / C1). No adapter. (§1.3)
4. **Wave 4 — self-host reuse: Qwythos 9B** on the local path; **GOT-OCR2.0** scheduled under P3-T8 OCR (Class C). (§1.4)

5. **Wave 5 — verify-then-reuse: Hyperbolic, Baseten** — fetch/confirm base URL at build, then fold into Wave-2-style reuse (Class F → A). (§1.2)
6. **Deferred / not-provider-layer (no rollout slot until unblocked): Subquadratic** (BLOCKED-until-GA, Class D); **OKF** (MCP resources) and **A2A** (protocol) tracked under their own streams (Class E).

OpenAI-compat reuse vs custom adapters: 13 confirmed hosted providers reuse the existing adapter config-only + GPT-Sol on the existing OpenAI adapter + 2 self-host on the existing local path; 0 net-new custom wire adapters among the confirmed extended set (OKF/A2A are non-provider protocol/context work; SubQ is blocked).

---

5. Composition footer — constitutional anchors touched

- **§11.4.6** (no-guessing) — every unverified base URL flagged `UNVERIFIED`; Perplexity tool-schema + xAI/Moonshot Anthropic-surface marked `UNCONFIRMED`; no endpoint invented (SubQ, Hyperbolic, Baseten).
- **§11.4.28** / **§CONST-045** / **§CONST-046** (decoupling / no-hardcoded-host / no-hardcoded-content) — a provider is a `{Name,BaseURL,APIKey-env}` config row, never a source literal.
- **§11.4.74** (extend-don't-reimplement) — reuse the existing OpenAI/Anthropic gateway + verifier `ProviderClient`; 0 net-new adapters.
- **§11.4.99** / **§11.4.150** (latest-source + deep multi-angle research) — every row cites the LATEST live doc read on 2026-07-06, ≥2 angles for the search-verified rows (official docs + integration guides/LiteLLM).
- **§11.4.108** / **§11.4.139** (four-layer runtime signature on a clean target) — the Class-A real-generate round-trip is the definition of done per provider.
- **§11.4.107(10)** (self-validated analyzer) — golden-good/golden-bad content-match analyzer, mutation-proofed.
- **§11.4.3** / **§11.4.69** (honest SKIP / sink-side evidence) — credential-absent → SKIP-with-reason; every PASS carries a captured wire artefact.
- **§11.4.21** (operator-blocked) — SubQ is OPERATOR-BLOCKED with enumerated unblock choices, not a faked PASS.
- **§11.4.50** / **§11.4.98** (determinism / full-automation) — Class-A re-runnable at `-count=3`, no manual step.
- **§11.4.42** / **§11.4.82** (priority-first / iteration-speedup) — cheapest-reuse-first rollout order.
- **CONST-036/037/038/039/040** — models from live `/v1/models` (no hardcoded list); freshness re-queue/poll; the CONST-039 already-integrated set is the reuse substrate; capability flags verifier-sourced.
- **CONST-039 gap-fill honesty** — GPT-Sol rides the existing OpenAI adapter (C1); Qwythos rides the existing local path — neither is a new provider.

Sources verified

Deep-research 2026-07-06: - https://creator.poe.com/docs/external-applications/openai-compatible-api - https://docs.perplexity.ai/getting-started/overview - https://console.sakana.ai/get-started - https://console.sakana.ai/models - https://platform.xiaomimimo.com/docs/en-US/api/chat/openai-api - https://cloud.tencent.com/document/product/1729/111007 - https://docs.x.ai/docs/overview - https://platform.kimi.ai/docs/api/chat - https://docs.z.ai/guides/overview/quick-start - https://docs.fireworks.ai/tools-sdks/openai-compatibility - https://docs.deepinfra.com/chat/overview - https://novita.ai/docs/guides/llm-api - https://docs.ai21.com/reference/jamba-1-6-api-ref - https://docs.reka.ai/chat/overview

In-tree source verified 2026-07-06: - `submodules/helix_llm/docs/API_CONTRACT.md` §2/§4/§5 (route inventory + OpenAI/Anthropic compat) - `submodules/llms_verifier/llm-verifier/providers/model_provider_service.go:45-49` (`ProviderClient{Name,BaseURL,APIKey}`) - `docs/research/07.2026/00_master/03_open_clarifications.md` (operator C1–C4 + resolved-without-operator) - `docs/research/07.2026/00_master/04_implementation_plan.md` P2-T1/P2-T4/P4-T1/P4-T2/P4-T3

Negative findings (§11.4.99(B)): - **Hyperbolic** and **Baseten** base URLs were NOT fetched from an authoritative page in this pass — recorded `UNVERIFIED` (§1.2), verify-at-build per `04_implementation_plan.md:90`; no endpoint asserted. - **Subquadratic** has no public API doc to fetch — `BLOCKED-until-GA` per operator C4; no endpoint invented. - **xAI** / **Moonshot** Anthropic-compat surfaces and **Perplexity** Sonar tool-calling schema were not confirmable on the fetched pages — marked `UNCONFIRMED`, to be probed at build (P2-T3); the OpenAI path each exposes IS confirmed and is the reuse target.

---

§11.4.138 honesty correction — provider-coverage claim in tag `helix-code-1.0.0-dev-0.0.1`

Correction (§11.4.138, filed 2026-07-08). An independent post-tag honesty audit found that the release **commit message** (`10c40c85`) and **tag annotation** for `helix-code-1.0.0-dev-0.0.1` list *"broad provider coverage"* undifferentiated alongside capabilities that DO carry live captured evidence (`docs/qa/<run-id>/RESULTS.md`) — embeddings, translation-NLLB, Whisper STT, Tesseract OCR, RAG, ACP→A2A, network-provider. This summary-layer phrasing was imprecise. (The substantive docs — this file, `RESUME.md`, and `06_providers_coverage/06_providers_coverage.md` — were audited **HONEST**: each correctly scopes

provider coverage as design/research/operator-gated. The over-claim is confined to the immutable commit-message + tag-annotation summary.)

Commit messages and tag annotations are immutable history — under §11.4.113 (absolute no-force-push / no history rewrite) they CANNOT be amended. This forward-correction record is therefore the honest mechanism (§11.4.6).

Precise scope of "broad provider coverage" as of this tag: it refers ONLY to (a) the pre-existing ~45-adapter roster in `helix_agent/internal/llm/providers/*` (unit-tested with mocked HTTP servers — NOT live), and (b) the extended-provider **design mapping** in this document (13 candidate providers researched + scoped as config-only reuse; **zero adapter code landed this session**). It should have carried the same **"(design)"** qualifier that "local vision (design)" correctly received in the same commit message.

No CONST-039 core-provider (OpenAI, Anthropic, Gemini, DeepSeek, Groq, Mistral, xAI, OpenRouter, Ollama, Llama.cpp) *live-proof harness* shipped in this tag — only a narrow ensemble probe + HelixLLM's own local live test existed. Per `RESUME.md` (§11.4.131, same commit), "broad-provider live proofs" are explicit NEXT / operator-gated work pending API keys (§11.4.10).

Tracked follow-up (in progress this session): a real per-provider CONST-039 live-proof harness — honest per-provider `SKIP: no-key` when keys are absent (§11.4.3), nonce-challenged real-call assertion when present, evidence under `docs/qa/<run-id>/provider_coverage/` — is being implemented as the immediate follow-up. No future claim of "provider coverage: reviewed GO" will be made without an accompanying `docs/qa/<run-id>/provider_coverage/RESULTS.md` evidence directory.